

Neural Taint Tracking for Prompt-Injection-Resistant Agents

Project Progress Report

Murtazin Aizat

June 26, 2026

1 Project Idea

Large language model (LLM) agents that call external tools (email, calendar, banking, file storage) routinely process *untrusted data*: tool outputs, retrieved documents, and third-party messages. Attackers can embed *indirect prompt injections* in such data—instructions that hijack the agent to perform malicious actions (e.g., exfiltrate email, delete files, or transfer funds) while the user requested something else.

Existing defenses such as **StruQ** separate instructions from data at the level of a *single* LLM query. Real agents, however, operate in *multi-step* loops (read inbox → reason → call `send_email`). Trust labels do not propagate through this loop by default.

Project goal: build an orchestration-layer **taint-tracking wrapper** around an existing agent that:

- labels each context fragment as *trusted* or *untrusted*;
- audits arguments of **privileged** tool calls before execution;
- blocks or sanitizes calls when untrusted content influences dangerous actions.

We do **not** train a new LLM; we add a policy layer on top of a standard tool-calling agent. Evaluation uses the **AgentDojo** benchmark (NeurIPS 2024).

2 Progress So Far

- Defined project scope: symbolic taint wrapper for the course project (~1.5 months); extension with a learned sanitizer and full evaluation planned for master’s thesis work.
- Set up the repository (`tesis/`): Python virtual environment, AgentDojo, baseline configuration, and automation scripts (`run_t0_baseline.ps1`, `collect_t0_results.py`).
- Completed **T0 baseline** (no defense) on the AgentDojo `workspace` suite (v1.2.2) with `gpt-4o-mini-2024-07-18` (via OpenRouter), attack `important_instructions`, 10 user tasks and $10 \times 14 = 140$ security cases.
- Produced reproducible artifacts: metric summaries, JSON traces, and per-case markdown logs under `results/`.

Note on model choice: the backbone LLM may change later (e.g., cost, availability, or capability). For any ablation, all treatments (T0–T4) will use the *same* model; if we switch models, T0 will be re-run to keep comparisons fair.

Table 1: AgentDojo workspace baseline without defense.

Metric	Value
Benign Utility	100% (10/10 user tasks)
Utility Under Attack	20.0% (28/140 cases)
Security rate (attacks blocked)	24.3% (34/140)
Targeted ASR	75.7% (106/140)
Injection goals achievable when asked directly	7/14

2.1 Baseline (T0) Results

Interpretation: without defense the agent is highly vulnerable (targeted attack success rate $\approx 76\%$). Benign utility remains high (100%), so the agent is capable but fails to resist injections. This baseline is the reference point for subsequent treatments (T1–T4).

Example (from traces): for user task “Who is invited to the Networking event?”, an injection hidden in a calendar event description instructs the agent to delete a file; the agent calls `delete_file`—attack success while the user task may still pass.

3 Theoretical Formulation

3.1 Threat Model

- **Trusted sources:** system prompt, user task, tool schemas.
- **Untrusted sources:** tool outputs, RAG chunks, memory entries derived from untrusted data.
- **Privileged sinks:** tools that modify external state (`send_email`, `delete_file`, `transfer_money`, etc.).

3.2 Symbolic Taint Model

Each text span s carries a trust label $\text{trust}(s) \in \{\text{trusted}, \text{untrusted}, \text{derived_untrusted}\}$ and a provenance identifier (e.g., which tool output it came from).

Propagation rules (deterministic):

- any tool output $\rightarrow$ **untrusted**;
- copying or quoting untrusted text into tool arguments $\rightarrow$ **derived_untrusted**.

3.3 Gate Policy

Before executing a privileged tool f with arguments $\mathbf{a}$:

$$\exists \text{arg} \in \mathbf{a} : \text{trust}(\text{arg}) \neq \text{trusted} \Rightarrow \text{block}$$

(with optional sanitization in later treatments).

3.4 Hypothesis and Metrics

Hypothesis: explicit provenance tracking and gating on privileged arguments reduces **Targeted ASR** while keeping **Benign Utility** acceptable.

Metrics (from AgentDojo): Benign Utility, Utility Under Attack, Targeted ASR; ablation over treatments $T0 \rightarrow T1 \rightarrow T2 \rightarrow T3$.

Formal noninterference guarantees are a longer-term goal; the current phase uses operational metrics and trace analysis.

4 Practical Formulation

4.1 Platform

AgentDojo provides a simulated workspace environment, user tasks, injection goals, attack templates, and automated checkers. Our defense integrates as a `BasePipelineElement` in the agent pipeline, intercepting tool calls before execution.

4.2 Planned Code Structure

```
src/taint/      -- models, propagation, gate, sanitizer
src/agent/     -- AgentDojo pipeline integration
eval/         -- benchmark runner, metrics
configs/       -- privileged_tools.yaml
```

4.3 Experimental Treatments

Treatment	Description	Status
T0	Baseline, no defense	Done
T1	Taint tags + traces, gate off	Next
T2	Strict gate on privileged tools	Planned
T3	Gate on sensitive fields only	Planned
T4	Rule-based sanitizer	Optional

4.4 Evaluation Subset

Primary ASR will be reported on the **7 injection tasks** (out of 14) that the agent can complete when the attack goal is given directly as the user prompt; the full 140 cases serve as supplementary results. This avoids conflating “attack blocked” with “attack goal too hard for the agent.”

Stack: Python 3.11+, AgentDojo, OpenAI-compatible API, pytest.

5 First Implementation Step

T1 — Tag-only layer (immediate next step after baseline):

1. Implement `TaintedSpan`, a taint registry, and symbolic propagation rules.
2. Integrate as a `BasePipelineElement` in the AgentDojo pipeline.
3. Keep the gate **disabled**; log `TaintTrace` JSON per agent step.
4. Re-run the same evaluation subset: ASR should remain $\approx$ T0; traces should expose taint leakage into tool arguments.
5. Add unit tests for synthetic cases (e.g., email body $\rightarrow$ `send_email(to=...)`).

Completion criterion for T1: traces correctly show provenance lineage; the system is ready to enable the gate in T2.